Memory-Access Optimizations and Sparse Compressed Representations in Matrix Multiplication

Dennis Mertens
Big Data (2026–2027) — Individual Assignment 2
GitHub Repository: https://github.com/d-mertens/big-data-assignments

***Abstract*—Standard dense matrix multiplication using the i-j-k loop order runs slowly on modern processors because reading down the columns of matrix B jumps through memory in steps of n elements, which can result in repeatedly missing the CPU cache. Here, I examine two techniques to improve performance: loop optimizations for dense matrices and sparse matrix formats for matrices with mostly zeros.**

For dense multiplication, reordering the loops to an i-k-j access pattern changes the inner loop to read memory sequentially with stride 1. At $n = 2048$, this makes dense multiplication $17.14\times$ faster than the i-j-k baseline, dropping runtime from $40.99\,$s to $2.39\,$s. It also runs faster than 64×64 cache-blocked tiling ($5.27\,$s) because simple sequential reads work better with the prefetcher.

For sparse matrices, I implement Gustavson's sparse matrix-matrix multiplication (SpGEMM) using the Compressed Sparse Row (CSR) format with a row accumulator. I test densities from 0.1% ($d = 0.001$) up to 50% ($d = 0.50$) for matrices up to $n = 2048$. At 99.9% sparsity ($d = 0.001$), CSR SpGEMM runs in $35\,\mu$s compared to $2.39\,$s for the optimized dense C code (a $68{,}343\times$ speedup) and uses 99.82% less memory.

Usually in theory, CSR uses less memory than a dense array as long as density stays below 66.7% ($d < 2/3$). But here in practice, sparse multiplication stays faster than dense multiplication up to around 50% to 60% density, mostly because multiplying sparse matrices produces many new non-zero entries in the output matrix.

I. Introduction and Objectives

Matrix multiplication ($C = A \times B$ where $C_{ij} = \sum_{k=1}^{n} A_{ik}B_{kj}$) is a very common numerical calculation in various fields of computer science [1]. In theory, multiplying two $n \times n$ matrices takes $\Theta(n^3)$ operations. In practice, how fast the calculation runs on a modern computer depends heavily on how memory is accessed and how well the CPU cache is used.

In Assignment 1, I compared the standard triple-loop algorithm across C, Java, and Python. The results showed that while C is fast for smaller sizes, execution slows down once matrices no longer fit into the CPU cache. This happens because the standard algorithm reads a matrix B column by column. Since C stores matrices row by row, jumping down a column means skipping n elements at each step, which can cause cache misses.

In this assignment, following the course specification [2], [3], I look at two ways to address this issue:

1) **Dense Optimizations:** Changing the loop order to i-k-j and applying cache-blocked tiling ($B = 64$) so the CPU reads memory sequentially and reuses data that is already in cache.
2) **Sparse Matrix Multiplication:** When matrices contain mostly zeros, storing and multiplying those zeros wastes both time and memory. I use the Compressed Sparse Row (CSR) format and Gustavson's SpGEMM algorithm to skip all zero values.

A. Research Questions

- *RQ1:* How much faster does dense multiplication run when switching from column-strided access (i-j-k) to sequential row access (i-k-j) and cache-blocked tiling as matrix sizes exceed cache capacity?
- *RQ2:* How does CSR SpGEMM scale in execution time and memory across different sparsity levels (from 99.9% down to 50% zeros)?
- *RQ3:* At what matrix density does sparse multiplication stop being faster or more memory-efficient than the best dense implementation?

B. Hypotheses

1) Loop reordering to i-k-j will give a speedup over the i-j-k baseline because the inner loop reads B and C with step size of 1.
2) For sparse matrices with few non-zeros ($d \leq 0.05$), CSR SpGEMM will be faster and use less memory than dense multiplication.

II. Experimental Setup

A. Hardware and Compiler

- **CPU:** Apple M4, 10 cores (4 Performance cores up to 4.4 GHz, 6 Efficiency cores).
- **Memory:** 128 KB L1 data cache per performance core, 16 MB shared L2 cache, 24 GB unified memory.
- **Compiler:** Apple Clang 21.0.0 with flags:

```
-O3 -fno-vectorize -std=c99 -Wall
```

I turned off automatic vectorization (`-fno-vectorize`) so the results show the memory access patterns directly (without added SIMD instructions).

B. Measurement and Verification

- **Timing:** Measured with `clock_gettime(CLOCK_MONOTONIC)` (essentially nanosecond resolution). Only the multiplication loop was timed and memory allocation and matrix setup were kept outside the timer.

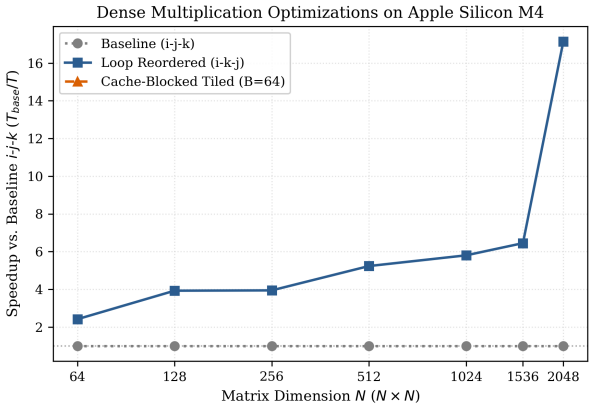


Fig. 1. Speedup of dense multiplication optimizations over the baseline i-j-k implementation.

- **Repetitions:** 5 timed runs per test and preceded by 1 untimed warm-up run (to prime the cache and memory pages).
- **Reproducibility:** Matrices were filled using a 64-bit Linear Congruential Generator (LCG) with fixed seeds (123456789 and 987654321).
- **Correctness Tests:** Every implementation passed 2×2 tests and was checked against standard dense multiplication to confirm numerical correctness.

III. Part A: Dense Matrix Multiplication Optimizations

A. Memory Layout and Stride Effects

In C, a 2D matrix is stored as one continuous block of memory in row-major order. That means an element $M_{i,j}$ sits at offset $i \cdot n + j$. When the CPU reads data from main memory, it loads a 64-byte cache line containing 8 contiguous double-precision numbers (8×8 bytes).

1) Baseline i-j-k Loop Order: In the standard algorithm, the innermost loop increments k:

$$C_{i,j} = \sum_{k=0}^{n-1} A_{i,k} \cdot B_{k,j} \tag{1}$$

Reading row i of A moves step by step through memory, but reading column j of B jumps by n elements at every single step. For large matrices ($n \geq 512$), this results in each read from B loading a 64-byte line from memory, using only one 8-byte number, and then discarding the remaining 7 numbers before they can be reused. So, this results in continuous cache misses.

2) Loop Reordering: i-k-j Algorithm: By swapping the two inner loops so that j becomes the innermost loop, the calculation becomes:

$$C_{i,j} \mathrel{+}= A_{i,k} \cdot B_{k,j} \quad \text{for } j = 0, \ldots, n-1 \tag{2}$$

In this arrangement:

- The number $A_{i,k}$ does not change during the inner loop and stays in a CPU register.

TABLE I
Dense Matrix Multiplication Runtime (compiled with Clang -O3 -fno-vectorize)

n	Baseline i-j-k Median [s]	Reordered i-k-j Median [s]	Tiled ($B = 64$) Median [s]	Speedup (i-k-j vs Base)
64	0.000179	0.000074	0.000232	2.42×
128	0.002228	0.000567	0.002024	3.93×
256	0.020551	0.005208	0.015152	3.95×
512	0.208964	0.039908	0.086622	5.24×
1024	1.757119	0.302719	0.697463	5.80×
1536	6.548821	1.015170	2.317265	6.45×
2048	40.993368	2.392013	5.266039	**17.14**×

- Both $B_{k,j}$ and $C_{i,j}$ are read and written across index j with a stride of 1, meaning the elements are contiguous in memory.

Every cache line loaded for B and C is now used 8 times in a row. The CPU hardware prefetcher also detects the sequential reads and pre-loads upcoming cache lines before the code asks for them.

3) Cache-Blocked Tiling ($B = 64$): To keep data inside the cache when matrices are large, I also implemented cache-blocked tiling [4]. The matrices are split into smaller blocks of size 64×64:

$$B = 64 \implies 64 \times 64 \times 8\,\text{bytes} = 32\,\text{KB} \tag{3}$$

Three blocks (A_{tile}, B_{tile}, C_{tile}) take 96 KB, which fits inside the 128 KB L1 data cache of each M4 core. Inside each tile, an i-k-j loop order is used to keep the stride 1 access.

B. Dense Experimental Results

Table I and Figure 1 show the measured runtimes as medians of 5 runs and the speedup factors.

C. Findings and Discussion

The measurements show performance differences across the three dense methods:

- At $n = 512$, i-k-j is 5.24× faster than the baseline.
- At $n = 2048$, three dense matrices take $3 \times 2048^2 \times 8\,\text{bytes} = 96\,\text{MB}$, which is larger than the 16 MB shared L2 cache. The baseline i-j-k runtime jumps to 40.99 s because it constantly has to wait on main memory.
- The reordered i-k-j implementation finishes in 2.39 s. This results in a speedup of 17.14×.

An interesting finding is that simple loop reordering (i-k-j) ran faster than 64×64 tiling (2.39 s vs. 5.27 s at $n = 2048$). The hardware prefetcher was very effective at streaming long rows of 2048 elements (16 KB at a time), so the sequential row streaming achieved better performance than tiled blocking.

IV. Part B: Sparse Matrix Multiplication

A. Compressed Sparse Row (CSR) Format

When matrices contain many zeros, storing all numbers in a 2D grid wastes memory and multiplying zeros ($0 \times x = 0$) wastes CPU time.

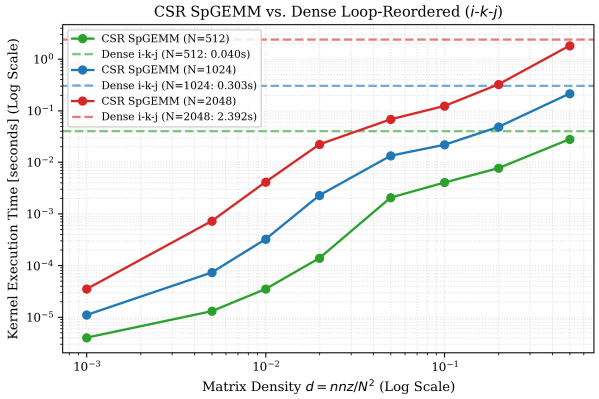


Fig. 2. Runtime of CSR SpGEMM across matrix densities compared to dense baselines ($n \in \{512, 1024, 2048\}$) on a log-log scale.

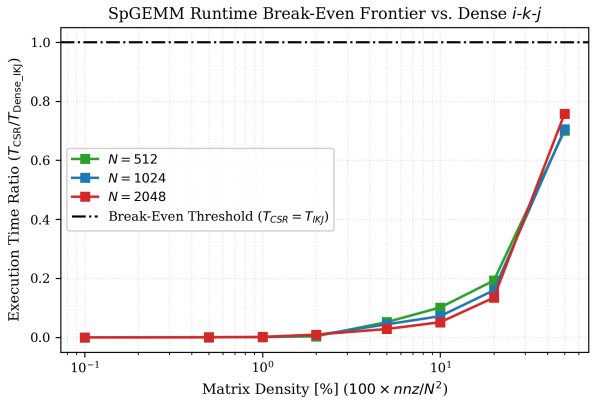


Fig. 3. Runtime ratio ($T_{\text{CSR}}/T_{\text{Dense_IKJ}}$) across densities. Values below 1.0 mean CSR SpGEMM runs faster than dense i-k-j.

The Compressed Sparse Row (CSR) format [5], [6] stores only the non-zero elements using three 1D arrays:

1) `values` (nnz doubles, 8 bytes each): The non-zero values stored row by row.
2) `col_indices` (nnz integers, 4 bytes each): The column index for each non-zero value.
3) `row_ptr` ($n+1$ integers, 4 bytes each): Pointers showing where each row starts, with `row_ptr[n]` $= nnz$.

B. Memory Comparison and Break-Even Point

The total memory used by an $n \times n$ matrix in CSR format is:

$$M_{\text{CSR}} = 8 \cdot nnz + 4 \cdot nnz + 4(n+1) = 12 \cdot nnz + 4(n+1) \text{ bytes} \tag{4}$$

A standard dense 2D array always takes:

$$M_{\text{dense}} = 8 \cdot n^2 \text{ bytes} \tag{5}$$

Using density $d = nnz/n^2$, the memory ratio is:

$$\frac{M_{\text{CSR}}}{M_{\text{dense}}} = \frac{12dn^2 + 4(n+1)}{8n^2} \approx 1.5d \tag{6}$$

Setting this ratio to 1.0 gives the theoretical memory break-even density:

$$1.5d^* = 1.0 \implies d^* = \frac{2}{3} \approx 66.67\% \quad (\text{sparsity } s^* \approx 33.33\%) \tag{7}$$

When density is below 66.7%, CSR uses less memory than a dense array. When density is above 66.7%, the extra 4-byte column indices make CSR use more memory than dense storage.

C. Gustavson's SpGEMM Algorithm

Multiplying two sparse matrices $C = A \cdot B$ is challenging because we do not know in advance where the non-zero entries of C will land. I used Gustavson's row-based SpGEMM algorithm [7], which computes C one row at a time:

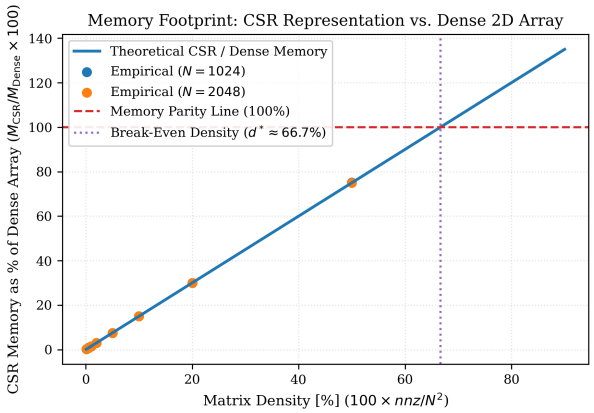


Fig. 4. Memory footprint of CSR vs. dense array across densities. This confirms the theoretical break-even density at $d^* \approx 66.7\%$.

1) For row i of A, iterate only over the non-zero entries $A_{i,k}$.
2) For each $A_{i,k}$, loop through row k of B and multiply $A_{i,k} \cdot B_{k,j}$.
3) Add the results directly into a temporary array `accum[j]`.
4) A helper array `marker[j]` tracks which columns have already been touched in row i and an `active_cols` list keeps track of occupied indices.
5) Once row i is finished, the non-zeros from `accum` are written into matrix C, and `accum` is reset for the next row.

This method avoids searching through columns and runs in $O(nnz(A) + \text{flops})$ time.

D. Experimental Results

Table II shows the performance of CSR SpGEMM at $n = 2048$ for densities from 0.1% ($d = 0.001$) to 50% ($d = 0.50$). Figures 2, 3, and 4 show the runtime comparisons, break-even curves and memory usage.

TABLE II
CSR SPGEMM RUNTIME AND COMPARISON WITH DENSE AT $n = 2048$

Density d	Sparsity $1-d$	SpGEMM Median [s]	CSR Mem [KB]	Mem Savings	Speedup vs Dense i-k-j
0.001	99.9%	0.000035	116	99.82%	**68,343×**
0.005	99.5%	0.000725	508	99.22%	**3,299×**
0.010	99.0%	0.004149	1,000	98.47%	**576×**
0.020	98.0%	0.022211	1,979	96.98%	**107×**
0.050	95.0%	0.067753	4,932	92.47%	**35.3×**
0.100	90.0%	0.123052	9,849	84.97%	**19.4×**
0.200	80.0%	0.321737	19,671	69.98%	**7.43×**
0.500	50.0%	1.813037	49,174	24.97%	**1.32×**

E. Findings

The benchmark results show three main insights:

1) Large Speedups at High Sparsity: When matrices have few non-zeros ($d \leq 0.01$, or 99% sparse), SpGEMM runs in a few milliseconds (4.15 ms at $d = 0.01$, $n = 2048$). This is $576\times$ faster than the optimized dense i-k-j code and $9{,}880\times$ faster than the baseline i-j-k code. At $d = 0.001$, the speedup reaches $68{,}343\times$.

2) Sparsity of the output matrix: When multiplying two sparse matrices, the output matrix C usually ends up less sparse than the input matrices. At $n = 2048$ with $d = 0.01$, matrices A and B have around 42,000 non-zeros each (1% density). However, their product C contains 779,173 non-zeros, an $18.6\times$ increase in non-zeros.

By $d = 0.10$ (10% non-zeros in A and B), matrix C contains $4{,}194{,}304$ non-zeros, meaning it is 100% full. Even though the output is fully dense, Gustavson's SpGEMM finishes in 0.123 s, which is still $19.4\times$ faster than the dense i-k-j kernel (2.39 s). This happens because the algorithm only loops over the 10% non-zero entries in A and B.

3) Runtime Break-Even Point: As density rises to $d = 0.50$, SpGEMM runtime increases to 1.81 s, getting closer to the 2.39 s runtime of dense i-k-j (ratio $T_{\text{CSR}}/T_{\text{Dense}} \approx 0.76$). Looking at the curve in Figure 3, the runtime break-even point occurs around $d \approx 55\% - 65\%$. Above this density, the extra work of aspects such as checking indices, following pointers and resizing buffers takes more time than what is saved by skipping zeros.

V. LIMITATIONS

- The test matrices used uniformly distributed random non-zero positions. Real-world sparse matrices (such as in social network graphs) often have clustered structures, which can change cache performance.
- All benchmarks were run focusing on memory access patterns, so only on one CPU core and without SIMD vectorization, therefore not a real world scenario.

VI. CONCLUSIONS

This evaluation shows the following takeaways for matrix multiplication:

1) **Always use i-k-j for dense multiplication:** Swapping two loop lines to create continuous memory reads (with stride 1) gave a $17.14\times$ speedup at $n = 2048$, without requiring extra memory.
2) **Sequential streaming can be faster than tiling:** On processors with large L1 caches and fast hardware prefetchers, simple sequential row reads can run faster than cache-blocked tiling because they avoid extra loop overhead and keep memory streaming smoothly.
3) **When to use CSR SpGEMM:** Sparse CSR multiplication provides large speedups (from $35\times$ up to $68{,}000\times$) when density is low ($d \leq 0.05$). The memory break-even point is 66.7% density and the runtime break-even point is around $55\% - 65\%$. Therefore, if a matrix has more than 60% non-zero values, dense i-k-j is usually the better option.

REFERENCES

[1] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 3rd ed. Johns Hopkins University Press, 2013.
[2] ULPGC, "Individual Assignments: Matrix Multiplication," *Big Data Course Specification*, Universidad de Las Palmas de Gran Canaria, 2026–2027.
[3] ULPGC, "Big Data Course," https://mltechdrawer.github.io/BIGDATA/, [Accessed 30-Oct-2026].
[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann, 2025.
[5] Y. Saad, *Iterative Methods for Sparse Linear Systems*, SIAM, 2003.
[6] T. A. Davis, *Direct Methods for Sparse Linear Systems*. SIAM, 2006.
[7] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Transactions on Mathematical Software (TOMS)*, vol. 4, no. 3, pp. 250–269, 1978.